

■ 第八课：函数进阶与按键

8.1 单片机最小系统

8.2 函数的调用

8.3 函数的形式参数和实际参数

8.4 按键★-输入设备

8.1 单片机最小系统

■ 1、电源

单片机的电源系统选择直接决定了其供电稳定性、IO 口驱动能力和适用场景，两者的关键区别。

对比维度	5V 单片机系统（3.4V-5.5V）	3.3V 单片机系统（2.7V-3.6V）
核心优势	1. IO 口驱动能力强（可直接驱动 LED、继电器）；2. 兼容性好（适配传统 5V 外设，如 5V ADC、5V 传感器）；3. 抗干扰能力较强（电压裕量更大）	1. 功耗极低（适合电池供电场景，如物联网设备）；2. 适配先进制程芯片（40nm 及以下制程多为 3.3V 及更低电压）；3. 可直接对接 3.3V 外设（如 SD 卡、WiFi 模块、FPGA）
典型芯片	AT89C51、STC89C52（8 位经典单片机）	STM32F103、ESP32
适用场景	工业控制（如继电器控制、电机驱动）、传统家电控制、5V 外设密集的简易系统	物联网（如无线传感器节点）、便携式设备（如手持检测仪）、需要对接高速外设（如 SPI 通信）的系统

STC89C51RC/RD+ 系列 单片机简介

STC89C51RC/RD+ 系列单片机是宏晶科技推出的新一代超强抗干扰 / 高速 / 低功耗的单片机，指令代码完全兼容传统 8051 单片机，12 时钟 / 机器周期和 6 时钟 / 机器周期可任意选择，最新的 D 版本内部集成 MAX810 专用复位电路。

特 点：

1. 增强型 6 时钟 / 机器周期，12 时钟 / 机器周期 8051 CPU
2. 工作电压：5.5V - 3.4V（5V 单片机） / 3.8V - 2.0V（3V 单片机）

推荐工作条件：

		CT54S138/CT74S138			CT54LS138/CT74LS138			单位
		最小	额定	最大	最小	额定	最大	
电源电压 V_{CC}	54	4.5	5	5.5	4.5	5	5.5	V
	74	4.75	5	5.25	4.75	5	5.25	
输入高电平电压 V_{IH}		2			2			V
输入低电平电压 V_{IL}	54			0.8			0.7	V
	74			0.8			0.8	
输出高电平电流 I_{OH}				-1000			-400	μA
输出低电平电流 I_{OL}	54			20			4	mA
	74			20			8	

8.1 单片机最小系统

常用的数字电路系统的电源类别有**24V**、**12V**、**5V**、**3.3V**、**2.5V**、**1.8V**。

电源电压	核心用途	典型应用模块	选择逻辑
24V	高压功率供电（大电流、远距离传输）	工业 传感器 （如接近开关、光电传感器）、步进电机驱动器、大功率继电器	1. 远距离传输时电压损耗小；2. 适配工业标准（如 PLC 常用 24V 供电）
12V	中功率供电	直流电机 （如小型风扇、微型水泵）、LCD 显示屏背光、线性稳压器（ LDO ）输入	1. 功率需求中等，无需 24V 高压；2. 避免 5V 低压的大电流损耗
5V	通用数字外设供电	USB 设备（如 USB 传感器、U 盘）、 TTL 电平接口（如串口模块）、中小功率芯片（如 5V 逻辑门）	1. 兼顾驱动能力与通用性；2. 传统数字电路的“标准电压”
3.3V	主流芯片核心 / 外设供电	单片机核心（如 STM32 ）、 FPGA （如 Xilinx Artix ）、无线模块（如蓝牙、 WiFi ）	1. 平衡功耗与性能；2. 当前主流芯片的“默认电压”
2.5V/1.8V	高精度 / 低功耗核心供电	高速 ADC/DAC （如 16 位以上精度芯片）、 DDR 内存（如 DDR3 ）、先进制程 MCU （如 40nm 以下芯片）	1. 芯片制程越小，核心电压越低（减少漏电损耗）；2. 高精度模块需低电压降低噪声

8.1 单片机最小系统

■ 2、晶振



图 8-1 有源晶振实物图



图 8-2 无源晶振实物图

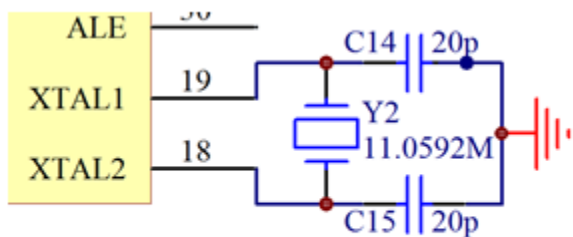


图 8-3 无源晶振接法

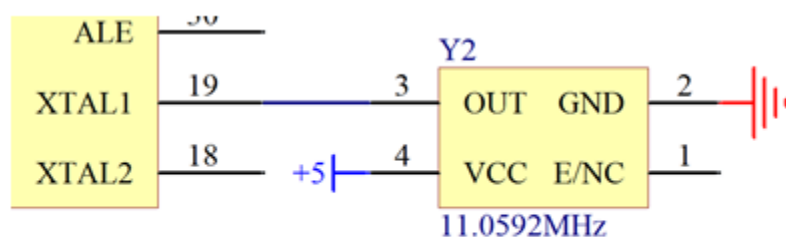


图 8-4 有源晶振接法

- 有源晶振（振荡器），震荡源独立，不依赖于外部电路。
1脚悬空或使能。优点：信号精度和质量比无源质量好，有源晶振依赖于电压值。
- 无源（晶体）晶振，接外部电路，加两个起振电容。
特点：不依赖于工作电压，可以震荡。

8.1 单片机最小系统

3、复位

静态

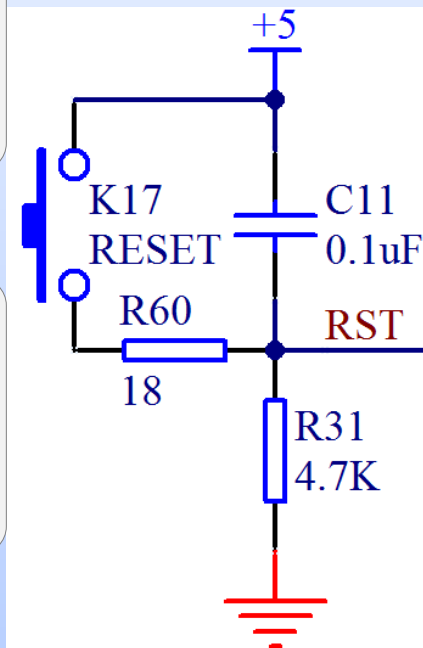
按键断开，电容开路，RST点的电位是低电平。单片机是高电平复位。

上电瞬间

电容两侧存在电压差，充电时电容相当于导线，支路上有电流， $RST=R31$ 的电压，随着充电过程，电流越来越小，RST电压随着电容充满后为电压低至0。复位时间计算 $t = 1.2RC$ ， $1.2 * 4700 * 0.0000001 = 564\mu s$ ，51单片机要求两个机器周期以上复位时间，11.0592M的机器周期1us，肯定是大于复位时间，满足复位。

手动复位

按下按键，左侧支路通路，RST的电压就是5V / $(18 + 4700) * 4700$ 接近5V，电容快速放电，18欧姆电阻消除电磁干扰，保持系统稳定。按下按键的时间，一般在100ms以上，复位时间足够。松开按键后，又给电容开始充电。



8.2 函数的调用

- **提高代码可读性：**每个函数专注于单一功能，通过函数名就能直观了解其用途，让代码结构更清晰，便于他人（或未来的自己）理解逻辑。
- **增强代码复用性：**封装后的功能可以在程序的多个地方被重复调用，避免重复编写相同代码，减少冗余。
- **便于维护和修改：**当需要修改某个功能时，只需调整对应的函数，无需在整个代码中寻找所有相关实现，降低了出错风险。
- **实现模块化开发：**不同函数可以由不同开发者独立编写和测试，便于团队协作，也使复杂问题分解为可管理的小任务。
- **简化测试过程：**单个函数可以被单独测试，更容易定位和修复问题，提高代码质量。
- **函数调用的一般形式：** 函数名(实参列表)
- **函数调用注意事项：**
 - 1、函数调用的时候**不需要加函数类型**。
 - 2、函数**调用之前必须进行定义或者声明**。
 - 3、函数**声明的时候必须加函数类型、函数的形参**，最后加上一个**分号表示结束**。

```

1 void main() {
2     unsigned int cnt = 0; //char最大255, 1ms*1000=1s, 最大要1000, 因此定义int型
3     unsigned long sec = 0;
4     unsigned char i = 0; //定义第几个数码管
5
6     ENLED = 0;
7     ADDR3 = 1;
8
9     TMOD = 0x01;
10    TH0 = 0xFC;
11    TLO = 0x67;
12    TRO = 1;
13
14    while(1)
15    {
16        if(TFO == 1)
17        {
18            TFO = 0;
19            TH0 = 0xFC;
20            TLO = 0x67;
21
22            cnt++;
23            if(cnt >= 1000)
24            {
25                cnt = 0;
26                sec++;
27                LedBuff[0] = LedChar[sec%10]; //显示个位
28                LedBuff[1] = LedChar[sec/10%10]; //显示十位
29                LedBuff[2] = LedChar[sec/100%10];
30                LedBuff[3] = LedChar[sec/1000%10];
31                LedBuff[4] = LedChar[sec/10000%10];
32                LedBuff[5] = LedChar[sec/100000%10];
33            }
34
35            PO = 0xFF;
36            switch(i)
37            {
38                case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; PO=LedBuff[0]; break;
39                case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; PO=LedBuff[1]; break;
40                case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; PO=LedBuff[2]; break;
41                case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; PO=LedBuff[3]; break;
42                case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; PO=LedBuff[4]; break;
43                case 5: ADDR2=1; ADDR1=0; ADDR0=1; i=0; PO=LedBuff[5]; break;
44                default: break;
45            }
46        }
47    }
48 }

```



```
43 void SecondCount ()
44 {
45     static unsigned int cnt = 0; //char最大255, 1ms*100
46     static unsigned long sec = 0;
47
48     cnt++;
49     if (cnt >= 1000)
50     {
51         cnt = 0;
52         sec++;
53         LedBuff[0] = LedChar[sec%10]; //显示个位
54         LedBuff[1] = LedChar[sec/10%10]; //显示十位
55         LedBuff[2] = LedChar[sec/100%10];
56         LedBuff[3] = LedChar[sec/1000%10];
57         LedBuff[4] = LedChar[sec/10000%10];
58         LedBuff[5] = LedChar[sec/100000%10];
59     }
60 }
```

```
61 void LedRefresh()  
62 {  
63     static unsigned char i = 0; //定义第几个数码管  
64  
65     P0 = 0xFF;  
66     switch(i)  
67     {  
68         case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; P0=LedBuff[0]; break;  
69         case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; P0=LedBuff[1]; break;  
70         case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; P0=LedBuff[2]; break;  
71         case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; P0=LedBuff[3]; break;  
72         case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; P0=LedBuff[4]; break;  
73         case 5: ADDR2=1; ADDR1=0; ADDR0=1; i=0; P0=LedBuff[5]; break;  
74         default: break;  
75     }  
76 }
```

```
20 void main() {  
21  
22     ENLED = 0;  
23     ADDR3 = 1;  
24  
25     TMOD = 0x01;  
26     TH0 = 0xFC;  
27     TL0 = 0x67;  
28     TR0 = 1;  
29  
30     while(1)  
31     {  
32         if(TF0 == 1)  
33         {  
34             TF0 = 0;  
35             TH0 = 0xFC;  
36             TL0 = 0x67;  
37             SecondCount();  
38             LedRefresh();  
39         }  
40     }  
41  
42 }
```

```
1 #include<reg52. h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 unsigned char LedChar[16]={
10     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
11     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
12 };
13 unsigned char LedBuff[6]={    //数组显示数码管值
14     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF
15 };
16
17 void SecondCount();
18 void LedRefresh();
19
20 void main() {
21
22     ENLED = 0;
23     ADDR3 = 1;
24 }
```

8.3 函数的形式参数和实际参数

- 实参：在调用一个有参数的函数时，函数名后边括号中的参数。
- 形参：在被调用的函数进行定义时，括号内的参数叫做形式参数。
- 1、函数定义中指定的形参，在未发生调用时不占用内存，函数调用时分配内存单元，调用结束释放内存。
- 2、实参可以是常量，也可以是简单或者复杂的表达式，但是要求必须有确定的值，在调用时传递给形参。 `c = add(1,a+b);`
- 3、形参必须制定数据类型，和变量定义一样。
- 4、形参和实参的数据类型应该相同或者赋值兼容。
- 5、主调函数在调用函数之前，应对被调函数做原型声明。
- 6、实参向形参的数据传递是单向传递，不能有形参再回传给实参。

```
1 unsigned char add(unsigned char x, unsigned char y);
2
3 void main()
4 {
5     unsigned char a=1;
6     unsigned char b=2;
7     unsigned char c=0;
8     c = add(a, b); //实参
9     while(1);
10 }
11
12 unsigned char add(unsigned char x, unsigned char y) //形参
13 {
14     unsigned char z =0;
15     z = x + y;
16     return z;
17 }
```


8.4 独立按键原理

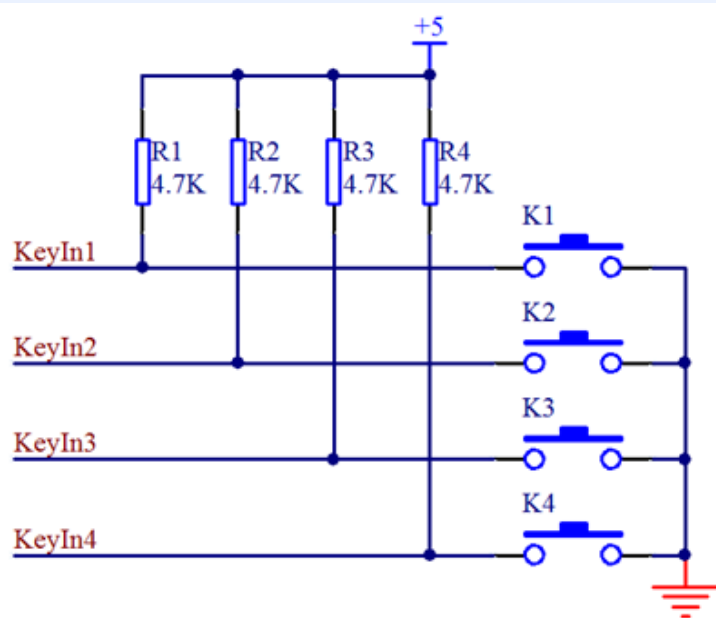


图 8-6 独立式按键原理图

按键按下时，KeyIn电平被拉低，是0V。读取单片机I/O口是高低电平来检测按键状态。

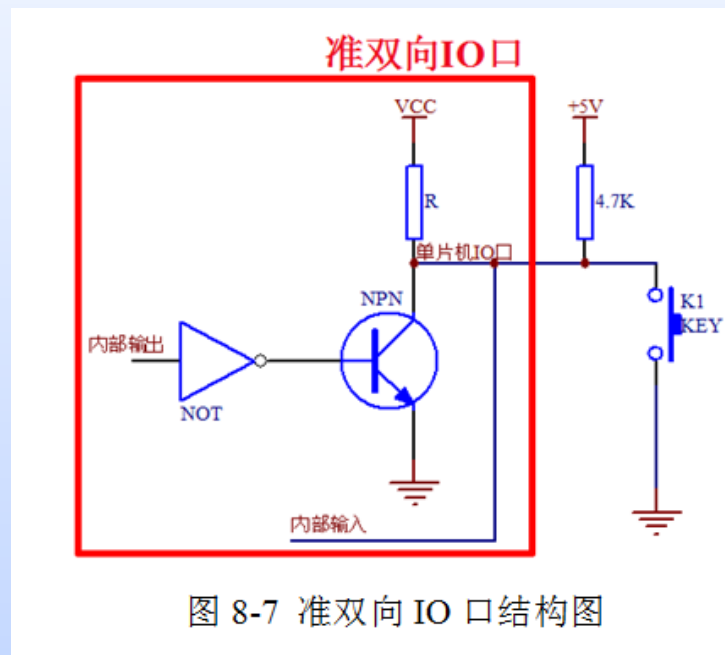


图 8-7 准双向 IO 口结构图

声明：单片机准双向IO口内部都有上拉电阻，单片机目前都用MOS管，不用三极管，为了方便理解，这里用三极管表示，原理几乎类似。

单片机读取外部信号，内部必须输出一个高电平。高电平取反，低电平，三极管截止。

按键按下，读取到低电平；按键弹起，读取到高电平。

矩阵按键原理

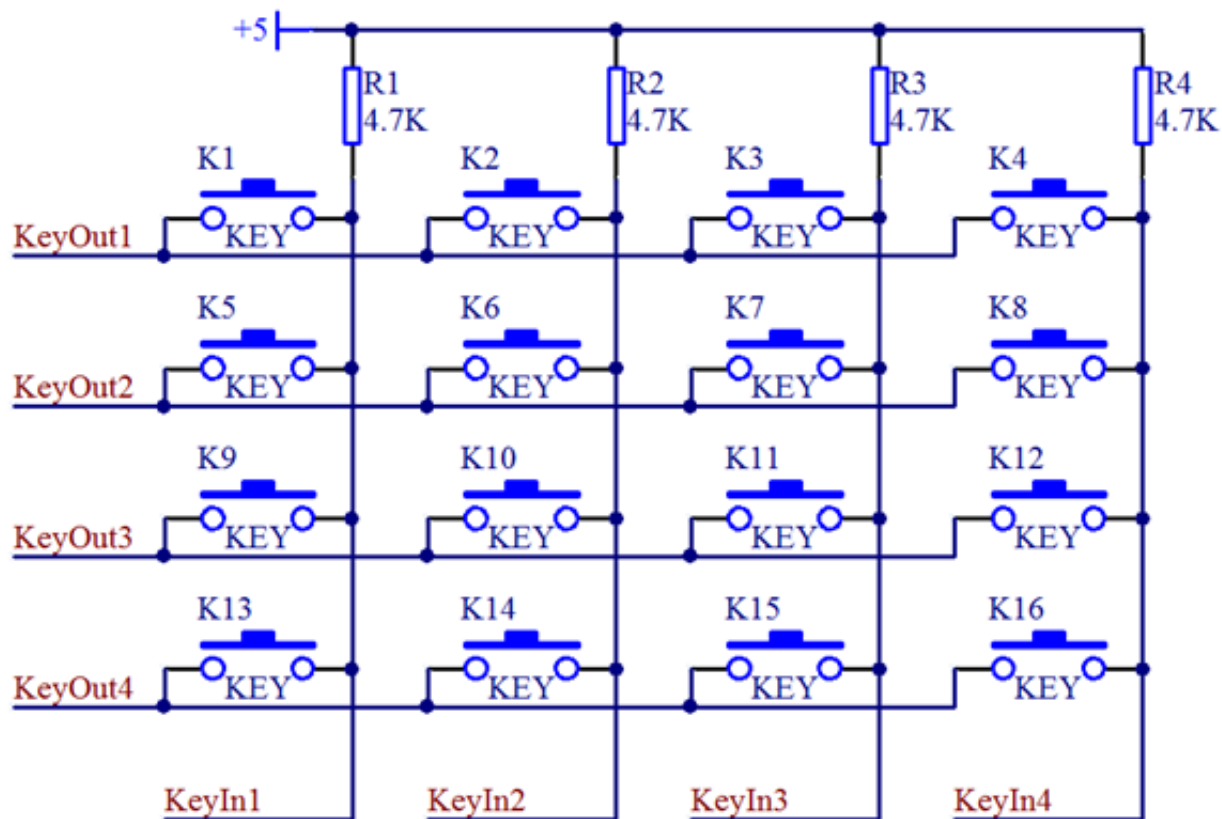
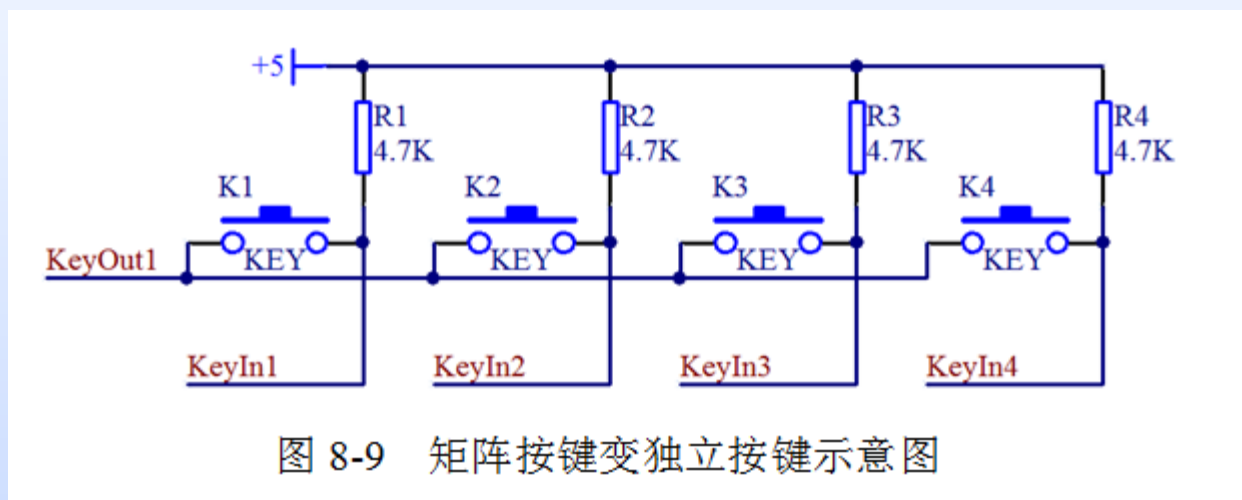


图 8-8 矩阵按键原理图

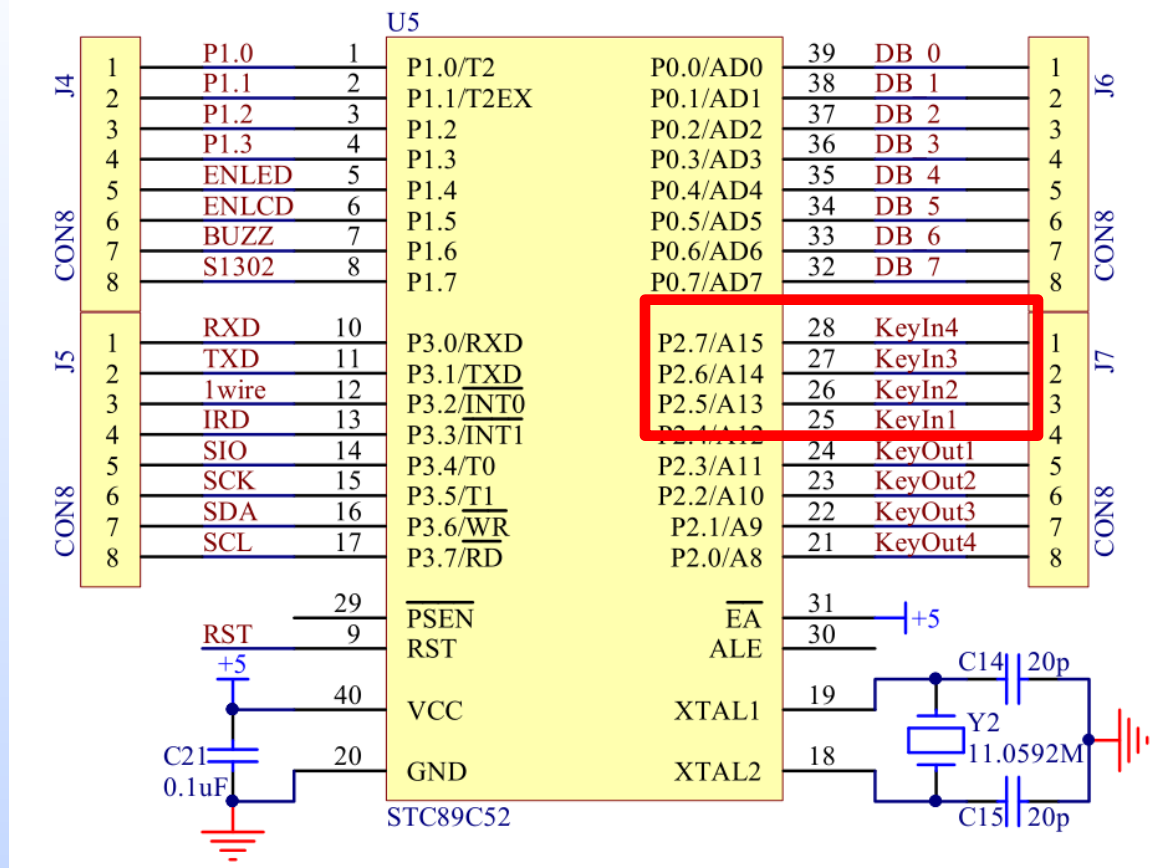
一个板子上要用很多按键，每个按键都占用I/O口，可以用矩阵按键更加方便。

矩阵按键变独立按键



KeyOut1低电平时， KeyOut2、3、4输出高电平
那么矩阵按键此时对应的就是4个独立按键。

独立按键是矩阵按键的特殊情况。



```

1 #include<reg52. h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 sbit LED9 = P0^7;
10 sbit LED8 = P0^6;
11 sbit LED7 = P0^5;
12 sbit LED6 = P0^4;
13
14 sbit KEY1 = P2^4; //
15 sbit KEY2 = P2^5;
16 sbit KEY3 = P2^6;
17 sbit KEY4 = P2^7;
18

```

开发板上按下按键对应的小灯亮

KeyOut1低电平

KeyOut2高电平

KeyOut3高电平

KeyOut4高电平

读取KeyIn1234的状态就可以

```

1#include<reg52.h>
2
3sbit ADDR0 = P1^0;
4sbit ADDR1 = P1^1;
5sbit ADDR2 = P1^2;
6sbit ADDR3 = P1^3;
7sbit ENLED = P1^4;
8
9sbit LED9 = P0^7;
10sbit LED8 = P0^6;
11sbit LED7 = P0^5;
12sbit LED6 = P0^4;
13
14sbit KEY1 = P2^4;//按下按键1234，对应小灯LED6789亮。
15sbit KEY2 = P2^5;
16sbit KEY3 = P2^6;
17sbit KEY4 = P2^7;
18
19void main()
20{
21    ENLED = 0;
22    ADDR3 = 1;
23    ADDR2 = 1;
24    ADDR1 = 1;
25    ADDR0 = 0;
26
27    P2=0xF7;//读取外部状态，必须是高电平输出，另外KeyOut1是低电平，其余高电平。
28
29    while(1)
30    {
31        LED9 = KEY1;
32        LED8 = KEY2;
33        LED7 = KEY3;
34        LED6 = KEY4;//IO口读取状态，状态由按键触发
35    }
36}

```

观察效果，按住按键1、2、3、4对应能让灯6、7、8、9点亮。实际应用时，我们希望按下按键，小灯点亮能持续，而并非一直按住按键，即：**按键扫描状态保存**。

Key4按键，按下一次数码管加1

可以把每次扫描的按键状态保存起来，把一次按键和下次按键扫描不同的状态，说明按键产生了动作。

记录扫描值：

上次扫描值是1，这次是0 说明按键弹起

上次扫描值是0，这次是1 说明按键按下

Key4按键来实现按下一次数码管加1

Key4按键，按下一次数码管加1-part1

```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 sbit LED9 = P0^7;
10 sbit LED8 = P0^6;
11 sbit LED7 = P0^5;
12 sbit LED6 = P0^4;
13
14 sbit KEY4 = P2^7; //只用到按键4
15
16 unsigned char code LedChar[]={
17     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
18     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
19 }; //数码管真值表
```

Key4按键，按下一次数码管加1-part2

```
21 void main()
22 {
23     bit backup = 1; //定义一个51特有的bit型变量，节省内存，0/1，弹起默认1
24     unsigned char cnt = 0;
25
26     ENLED = 0;
27     ADDR3 = 1;
28     ADDR2 = 0;
29     ADDR1 = 0;
30     ADDR0 = 0;
31
32     P2=0xF7; //读取外部状态，必须是高电平输出，另外KeyOut1是低电平，其余高电平。
33     P0=LedChar[cnt];
34
35     while(1)
36     {
37         if(KEY4 != backup) //按键发生变化，当前值与上一次的值不等
38         {
39             if(backup == 0) //按键之前是0，这次是1，按键弹起
40             {
41                 cnt++; //按键次数加1
42                 if(cnt >= 10)
43                 {
44                     cnt = 0; //只用一个数码管，最多加到10就清零
45                 }
46                 P0 = LedChar[cnt]; //计数值显示在数码管
47             }
48             backup = KEY4; //更新backup为当前值，以备下次比较
49         }
50     }
51 }
52 }
```

新问题：有时候一次加1，有时候加几个。

按键消抖

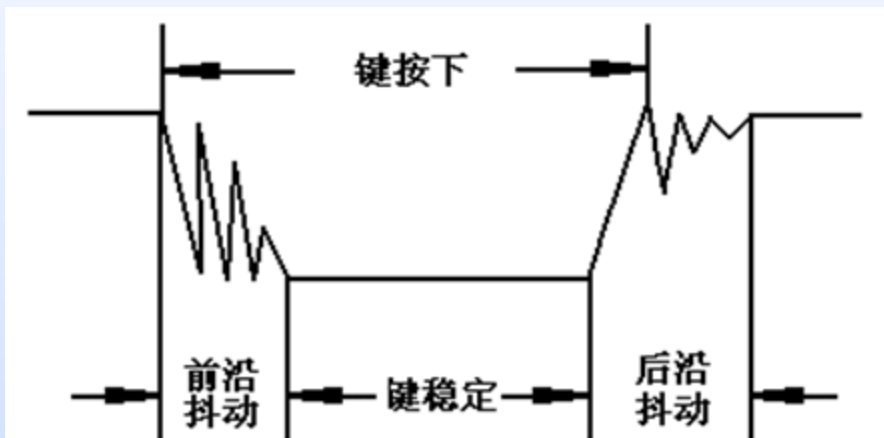


图 8-10 按键抖动状态图

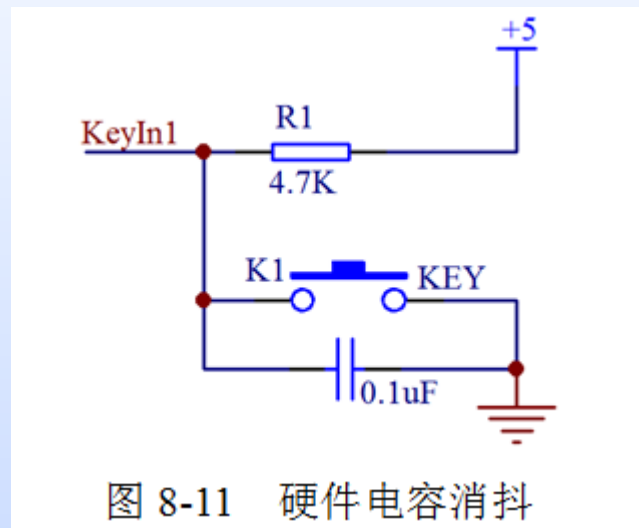


图 8-11 硬件电容消抖

按键本身特点：机械触点的弹性作用，按下和弹起会有抖动。机械触点断开、闭合，开关闭合时不会马上就稳定接通，在断开时也不会一下子彻底断开，而在闭合和断开的瞬间伴随一连串的抖动。

硬件消抖：电容可以滤除高频，抖动实际上就是高频毛刺，容易释放掉。实际上应用时，效果往往不好，增加成本，绝大多数用**软件消抖**。按下按键，延时10ms后再检测确认是否按下，按键弹起，延时10ms后再检测是否弹起。抖动在10ms以内，人按按键基本在100ms以上。

矩阵按键扫描程序-消抖part1

```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 sbit LED9 = P0^7;
10 sbit LED8 = P0^6;
11 sbit LED7 = P0^5;
12 sbit LED6 = P0^4;
13
14 sbit KEY4 = P2^7; //只用到按键4
15
16 unsigned char code LedChar[]={
17     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
18     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
19 }; //数码管真值表
20
21 void delay();
22 void main()
23 {
24     bit backup = 1; //定义一个51特有的bit型变量, 节省内存, 0/1, 弹起默认1
25     bit keybuf = 1; //保存前一次的扫描值, 按键值备份, 作为比较用
26     unsigned char cnt = 0;
27
28
29     ENLED = 0;
30     ADDR3 = 1;
31     ADDR2 = 0;
32     ADDR1 = 0;
33     ADDR0 = 0;
34
35     P2=0xF7; //读取外部状态, 必须是高电平输出, 另外KeyOut1是低电平, 其余高电平。
36     PO=LedChar[cnt];
37 }
```

矩阵按键扫描程序-消抖part2

```
38 while(1)
39 {
40     keybuf = KEY4; //读取KEY4
41     if(keybuf != backup) //KEY4和backup不同
42     {
43         delay();
44         if(keybuf == KEY4) //延时后再判断一次是否为抖动，若不是抖动
45         {
46             if(backup == 0) //
47             {
48                 cnt++;
49                 if(cnt >= 10)
50                 {
51                     cnt = 0;
52                 }
53                 PO = LedChar[cnt]; //改变数码管的值
54             }
55             backup = keybuf; //案件发生变化backup重写
56         }
57     }
58 }
59
60 }
61 void delay()
62 {
63     unsigned int i = 1000;
64     while(i--);
65 }
```

按键消抖

实际做项目开发，程序量大，各种状态值很多，**while(1)**主循环要不停扫描各种状态值是否发生变化，及时进行任务调度，如果程序中加了延时操作，程序还在**delay**，延时完再去检查的时候，可能错过窗口期检测不到应该检测的事件发生。

为了避免上述情况，要尽量缩短**while (1)**循环一次所用的时间，那么长时间延时的操作必须借用其他方法处理。

对于按键的连续**16ms**的**8次**状态进行判断，如果全**0**则按下，全**1**则弹起，**01**交错就是抖动，不以理会。单片机：**检测状态，而非监测过程。**

1111111111111111111101001000000000000000000000000001001011111111111111111111

弹起 抖动 按下 抖动 弹起

图 8-12 按键连续扫描判断

```
1#include<reg52.h>
2
3sbit ADDR0 = P1^0;
4sbit ADDR1 = P1^1;
5sbit ADDR2 = P1^2;
6sbit ADDR3 = P1^3;
7sbit ENLED = P1^4;
8
9sbit KEY1 = P2^4;
10sbit KEY2 = P2^5;
11sbit KEY3 = P2^6;
12sbit KEY4 = P2^7;
13
14unsigned char code LedChar[]={
15    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
16    0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
17}; //数码管真值表
18
19bit KeySta = 1; //按键状态
```

```
21 void main()
22 {
23     bit backup = 1;
24     unsigned char cnt = 0;
25
26     ENLED = 0;
27     ADDR3 = 1;
28     ADDR2 = 0;
29     ADDR1 = 0;
30     ADDR0 = 0;
31
32     EA = 1; //使能总中断
33     TMOD = 0x01; //T0定时器模式1
34     TH0 = 0xF8; //初值对应2ms
35     TL0 = 0xCD;
36     ET0 = 1; //使能T0中断
37     TR0 = 1; //启动T0
38
39     P2=0xF7; //KeyOut1是低电平，其余高电平。
40     P0=LedChar[cnt];
41 }
```

```
44 while(1)
45 {
46     if(KeySta!= backup)
47     {
48         if(backup == 0)
49         {
50             cnt++;
51             if(cnt>=10)
52             {
53                 cnt = 0;
54             }
55             P0= LedChar[cnt];
56         }
57         backup = KeySta;
58     }
59 }
60 }
61 }
```

```
62
63 void InterruptTimer0() interrupt 1
64 {
65     static unsigned char keybuf = 0xFF; // 8次的扫描结果放在8位之中
66
67     TH0 = 0xF8; // 初值对应2ms
68     TL0 = 0xCD;
69
70     keybuf = (keybuf << 1) | KEY4; // 每进一次中断，从低位挤进来一个，高位挤出去一个，最后一位更新KEY4
71     if (keybuf == 0x00)
72     {
73         KeySta = 0;
74     }
75     else if (keybuf == 0xFF)
76     {
77         KeySta = 1;
78     }
79     else
80     {
81     }
82 }
```


矩阵按键的扫描

数码管动态显示-轮流切换读输出

矩阵按键-轮流切换读输入

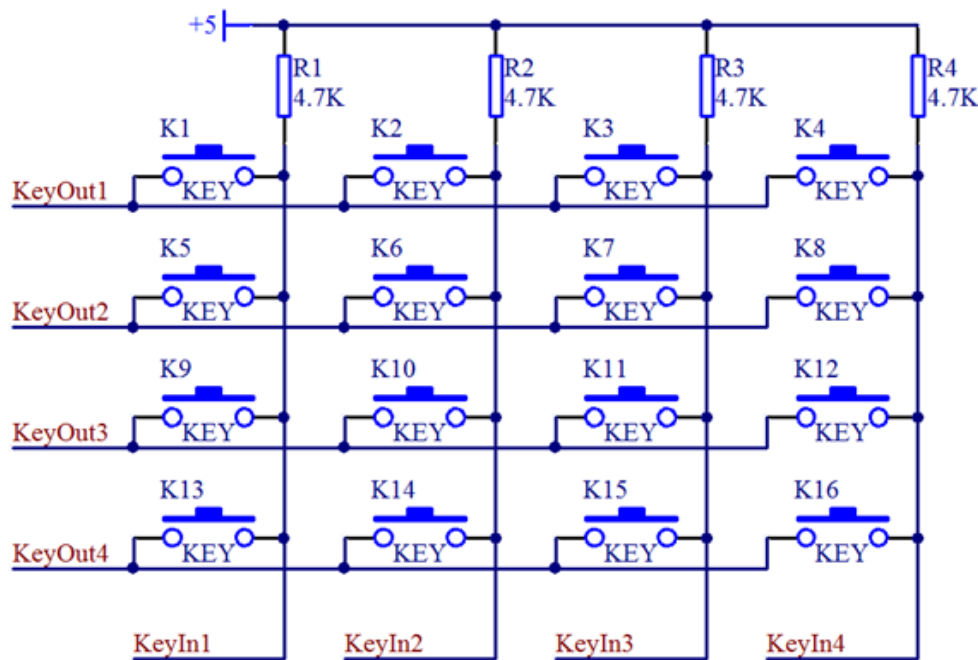


图 8-8 矩阵按键原理图

KeyOut1低电平时， KeyOut2、3、4输出高电平，按键读K1K2K3K4

KeyOut2低电平时， KeyOut1、3、4输出高电平，按键读K5K6K7K8...

矩阵按键的扫描

4个KeyOut输出，中断4次完成一次全部按键扫描

如果2ms进入一次中断，连续8次按键状态扫描，

4行就需要 $2*8*4=64\text{ms}$ ，扫描时间太长。

故改用1ms中断判断4次采样值， $1*4*4=16\text{ms}$ 。

16ms依然满足消抖时间要求。

矩阵按键通过所在行和所在列进行编号。

```
1 #include<reg52. h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 sbit KEY_IN_1 = P2^4;
10 sbit KEY_IN_2 = P2^5;
11 sbit KEY_IN_3 = P2^6;
12 sbit KEY_IN_4 = P2^7;
13
14 sbit KEY_OUT_1= P2^3;
15 sbit KEY_OUT_2= P2^2;
16 sbit KEY_OUT_3= P2^1;
17 sbit KEY_OUT_4= P2^0;
18
19 unsigned char code LedChar[]={
20     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
21     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
22 };//数码管真值表
23
24 unsigned char KeySta[4][4] ={
25 {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}
26 };//按键状态, 数组不能用bit变量定义
27
```

```
28 void main()
29 {
30     unsigned char i, j;
31     unsigned char backup[4][4] = {
32         {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}, {1, 1, 1, 1}
33     };
34     unsigned char cnt = 0;
35
36     ENLED = 0;
37     ADDR3 = 1;
38     ADDR2 = 0;
39     ADDR1 = 0;
40     ADDR0 = 0;
41
42     EA = 1; //使能总中断
43     TMOD = 0x01; //T0定时器模式1
44     TH0 = 0xFC; //初值对应2ms
45     TL0 = 0x67;
46     ET0 = 1; //使能T0中断
47     TR0 = 1; //启动T0
48
49     P0 = LedChar[0];
50 }
```

```
51 while(1)
52 {
53     for(i=0;i<4;i++)//i代表out
54     {
55         for(j=0;j<4;j++)//j代表in
56         {
57             if(backup[i][j]!=KeySta[i][j])//有按键按下
58             {
59                 if(backup[i][j]!=0)
60                 {
61                     P0 = LedChar[i*4+j];//数字上就是0-F按键编号
62                 }
63                 backup[i][j] = KeySta[i][j];
64             }
65         }
66     }
67 }
68 }
69 }
```

```

69
70 void InterruptTimer0() interrupt 1
71 {
72     static unsigned char keyout = 0; //8次的扫描结果放在8位之中
73     unsigned char i = 0;
74     static unsigned char keybuf[4][4] = { //16ms检测16个按键值暂存
75         {0xFF, 0xFF, 0xFF, 0xFF},
76         {0xFF, 0xFF, 0xFF, 0xFF},
77         {0xFF, 0xFF, 0xFF, 0xFF},
78         {0xFF, 0xFF, 0xFF, 0xFF}
79     };
80     TH0 = 0xFC; //初值对应2ms
81     TL0 = 0x67;
82
83     keybuf[keyout][0] = ( keybuf[keyout][0] << 1 ) | KEY_IN_1; //读按键，左移一位，保存最新的是KEY_IN1
84     keybuf[keyout][1] = ( keybuf[keyout][1] << 1 ) | KEY_IN_2;
85     keybuf[keyout][2] = ( keybuf[keyout][2] << 1 ) | KEY_IN_3;
86     keybuf[keyout][3] = ( keybuf[keyout][3] << 1 ) | KEY_IN_4;
87     for(i=0; i<4; i++)
88     {
89         if((keybuf[keyout][i]&0x0F)==0x00) //最新读的低四位如果都是0
90         {
91             KeySta[keyout][i]=0; //按下
92         }
93         else if((keybuf[keyout][i]&0x0F)==0x0F)
94         {
95             KeySta[keyout][i]=1; //弹起
96         }
97     }
98     keyout++; //切换keyout下一个
99     keyout = keyout & 0x03; //加4归0最后一行退出
100
101     switch(keyout)
102     {
103         case 0: KEY_OUT_4 = 1; KEY_OUT_1 = 0; break; //41, 1先拉高再清零
104         case 1: KEY_OUT_1 = 1; KEY_OUT_2 = 0; break; //12
105         case 2: KEY_OUT_2 = 1; KEY_OUT_3 = 0; break; //23
106         case 3: KEY_OUT_3 = 1; KEY_OUT_4 = 0; break; //34
107         default: break;
108     }
109 }

```

■ `keyout = keyout & 0x03;`

`0&0x03=0x00;`

`1&0x03=0x01;`

`2&0x03=0x02;`

`3&0x03=0x03;`

`4&0x03=0x00;`

```
if(keyout>=4)
```

```
{
```

```
keyout=0;
```

```
}
```


CASE	KEY_OUT_4	KEY_OUT_3	KEY_OUT_2	KEY_OUT_1
0	1	1	1	0
1	1	1	0	1
2	1	0	1	1
3	0	1	1	1

3-1: KEY_OUT_2、KEY_OUT_3不变
0-1: KEY_OUT_3、KEY_OUT_4不变
1-2: KEY_OUT_1、KEY_OUT_4不变
2-3: KEY_OUT_1、KEY_OUT_2不变

```
keyout++; //切换keyout下一个
keyout = keyout & 0x03; //加4归0最后一行退出
```

```
switch(keyout)
{
    case 0: KEY_OUT_4 = 1; KEY_OUT_1 = 0; break; //41, 1先拉高再清零
    case 1: KEY_OUT_1 = 1; KEY_OUT_2 = 0; break; //12
    case 2: KEY_OUT_2 = 1; KEY_OUT_3 = 0; break; //23
    case 3: KEY_OUT_3 = 1; KEY_OUT_4 = 0; break; //34
    default: break;
}
```

作业

- 操作练习：
- 1、学会独立按键和矩阵按键的电路设计方法和软件编程思路。